
Infrastructure & Access Requirements

NAPCO Nucleus AI Agent — a memo for IT and leadership

Mohammad Kamrul Hasan • April 2026

NAPCO Nucleus AI Agent is a Claude-native automation platform built to take two day-to-day burdens off the team: turning client communications into actionable backlog items, and continuously validating the MVP Access platform through automated tests. The agent already runs seven scheduled workflows on a self-hosted Windows VM today.

This short memo lists the infrastructure and access we need from IT and operations to take the system from its current developer-run state into properly supported daily operations.

1. DevOps & Release Automation (CI/CD)

Stable foundations for the nightly build-and-deploy pipeline.

The nightly TFS → MSBuild → IIS pipeline runs unattended at 22:00 BDT and primes the morning test cycle. If any of the items below moves without notice, the pipeline writes to nowhere and the next morning's tests all fail.

Dedicated Deployment Server (IIS Host)

- A stable IIS environment that will not be re-purposed without notice. The pipeline binds to a specific host; if it moves, every run after that fails until the configuration is reapplied.

Static UNC Path for Build Artifacts

- A fixed, persistent network path that the deploy step writes to. A documented, change-controlled path means any future move is planned, not a surprise that surfaces at 03:00 the next morning when E2E starts failing.

TFS Access — Dedicated Service Account

- **TFS_USERNAME / TFS_PASSWORD** — non-expiring credentials owned by the team, not tied to an individual.
- **TFS_URL** — the base collection endpoint.
- **TFS_PROJECT_PATH** — the branch the pipeline pulls from (for example, \$/MVPAccess/Main).
- Read access on the MVPAccess project is sufficient; the pipeline does not write back to TFS.

2. Test Automation Environment

Isolated and stable, so daily test results stay meaningful.

Three test suites (functional, integration, end-to-end) run nightly against staging, plus a weekly load suite. Their value depends on the environments staying predictable from one run to the next.

Staging Environment — Manual & Functional QA

- A permanent, documented URL that is not silently moved between sprints. The test scripts hard-code the staging endpoint; a change without notice breaks every regression run that night.
- Persistent test-data accounts with non-rotating credentials. When test accounts are reset, the suite fails overnight and we only find out the next morning.

Performance Sandbox — Load Testing

-
- A dedicated load-test target that is separate from the staging environment used by manual QA. Today they share, which means the weekly Locust run can slow down whatever testing the QA team is doing on the same box.

3. Project Management & Communication

The pipes that carry requirements in and tasks out.

The agent ingests requirements from email, Teams channels, and meeting recordings, splits them into ~3-hour atomic tasks, and pushes them to a backlog. Two pieces close that loop.

Centralized Backlog

- A single project-management tool that holds all user stories and technical tasks as the source of truth. Today the agent pushes 3-hour atomic tasks to GitLab. The ask is that we agree this is the single backlog and do not fragment work across multiple tools — fragmentation defeats the agent's dedup logic.

Power Automate License + Flow

- A Power Automate workflow that forwards Microsoft Teams channel posts into the IMAP allowlist that the agent already polls.
- The agent does not call the Microsoft Graph API directly. Power Automate is the bridge — without it, requirements raised in Teams never reach the agent's intake pipeline.

The logic for the AI-driven workflows (Requirement Management and Test Automation) is fully developed. Securing these infrastructure prerequisites is the final step required to move from the development phase to active, daily operations.

Happy to walk through any of this in a short conversation.

— Mohammad Kamrul Hasan